

```

0x000055555555480a <+96>: mov    %rdx,%rsi
0x000055555555480d <+99>: mov    %rax,%rdi
0x0000555555554810 <+102>: callq 0x555555554670 <strcmp@plt>
0x0000555555554815 <+107>: test   %eax,%eax
0x0000555555554817 <+109>: jne    0x555555554827 <main+125>
0x0000555555554819 <+111>: lea    0xcd(%rip),%rdi    # 0x5555555548ed
0x0000555555554820 <+118>: callq 0x555555554640 <puts@plt>
0x0000555555554825 <+123>: jmp    0x555555554833 <main+137>
0x0000555555554827 <+125>: lea    0xd7(%rip),%rdi    # 0x555555554905
0x000055555555482e <+132>: callq 0x555555554640 <puts@plt>
0x0000555555554833 <+137>: mov    $0x0,%eax

```

Figure 9: The disassembled code of *test2*

```

File Edit View Search Terminal Help

Breakpoint 2, 0x0000555555554810 in main ()
(gdb) info registers
rax            0x7fffffffde4a    140737488346698
rbx            0x0                0
rcx            0x7ffffffdd0560    140737351845216
rdx            0x7fffffffde50    140737488346704
rsi            0x7fffffffde50    140737488346704
rdi            0x7fffffffde4a    140737488346698
rbp            0x7fffffffdec0    0x7fffffffdec0
rsp            0x7fffffffde40    0x7fffffffde40
r8             0x0                0
r9             0x0                0
r10            0x0                0
r11            0x5555555548ec    93824992233708
r12            0x5555555546a0    93824992233120
r13            0x7fffffffdfa0    140737488347040
r14            0x0                0
r15            0x0                0
rip            0x555555554810    0x555555554810 <main+102>
eflags        0x206    [ PF IF ]
cs            0x33    51
ss            0x2b    43
ds            0x0                0
es            0x0                0
fs            0x0                0
---Type <return> to continue, or q <return> to quit---
gs            0x0                0
(gdb) x/s 0x7fffffffde4a
0x7fffffffde4a: "hello"
(gdb) x/s 0x7fffffffde50
0x7fffffffde50: "dummy"

```

Figure 10: Register content in *test2* with GDB

When the program stops at the second breakpoint, we will get the contents of the registers with the command *info registers*. Figure 10 shows the contents of the registers, as well as the contents of the registers being printed in string format with the command *x/s*, where the register *rdx* contains the test password *dummy* entered by us and the register *rax* contains the actual password *hello*.

Thus, from Figure 10, we can see that the password is *hello*. Hence, GDB has been successfully used to unearth the hidden password in the executable file *test2*. Please note that the techniques we have discussed here are quite simple. People who make difficult Crackme programs use a variety of

techniques to make password recovery very hard. Some of the techniques involve automatic key generation, obfuscating the code itself, etc. But if you want to practice further and be a master of software reverse engineering, there are a lot of resources available on the Internet. However, we would like to caution you to be extremely careful while downloading and using Crackme programs from unreliable

sources, as you would be downloading and running a program whose code is absolutely unknown to you. For all you know, it could be a virus or some sort of malicious program. So absolute care must be taken while practising with Crackme programs downloaded from the Internet. But all things said, we would like to share our personal experience. We have downloaded and used Crackme programs from a popular website called *crackmes.one* many times and never had any trouble with the executable files.

Before winding up, we would like to make one confession and give one warning. The confession is that the simple techniques we have discussed here will not help you break even the simplest puzzle available on the Internet. But the important question is: does there exist a really unbreakable Crackme program? Well, there cannot be. The program has to compare the password entered by the user with the actual password. For this, the real password must be stored somewhere in the program. It may be obfuscated or processed into some other form by an algorithm, but whatever be the technique employed, a trace of the password will be there in the program. So, a good programmer with sufficient skills and lots of patience can eventually break any Crackme program. So, in theory, every Crackme program is crackable but, in practice, many a programmer has gone mad trying to crack Crackmes! So, be careful. Our warning is that applying the knowledge you may have gained by reading this article and by further studies on proprietary software may not be always legal due to intellectual property rights. So you should be extra careful while choosing software for reverse engineering. **END** 🐧

By: Deepu Benson and Jerin Thomas

Deepu Benson is a free software enthusiast and his area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. The author maintains a technical blog at www.computingforbeginners.blogspot.in.

Jerin Thomas holds the post of assistant professor in Amal Jyothi College of Engineering, Kerala. His area of interest is digital image processing.